

# PRÉ- PROCESSAMENTO DE DADOS

Preparando os dados antes de treinar modelos

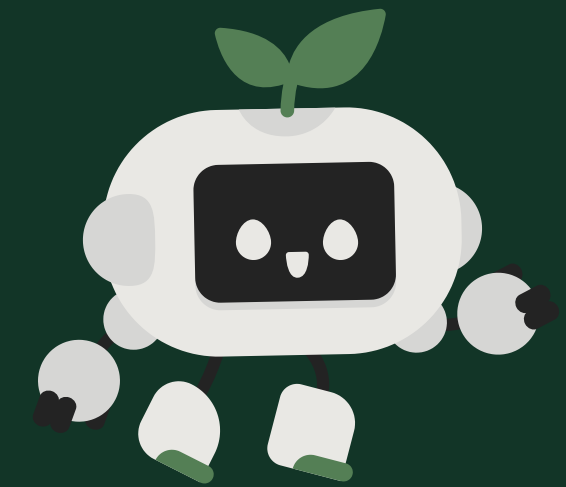


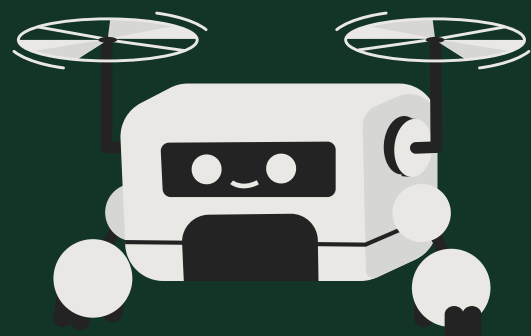
# OBJETIVOS DA SEMANA

## O que vamos aprender hoje?

- Entender por que o pré-processamento é a etapa mais importante (e mais demorada) de um projeto de ML
- Aprender a lidar com valores nulos (missing data)
- Conhecer escalonamento de variáveis: MinMaxScaler e StandardScaler
- Entender codificação de variáveis categóricas (texto -> números)
- Introduzir o conceito de Pipelines para organizar o fluxo de trabalho

**"Dados limpos e bem preparados → modelos melhores. Dados sujos → modelos ruins, não importa quão sofisticado seja o algoritmo."**

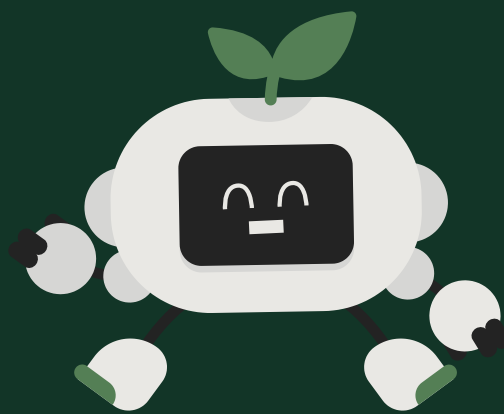




# POR QUE PRÉ-PROCESSAR OS DADOS?

## A regra de ouro do Machine Learning

"Garbage in, garbage out"  
(Lixo entra, lixo sai)



| Dados ruins                   | Dados bons         |
|-------------------------------|--------------------|
| Valores faltando              | Dados completos    |
| Valores em escalas diferentes | Dados padronizados |
| Texto no lugar de números     | Dados numéricos    |
| Outliers extremos             | Outliers tratados  |
| Inconsistências               | Dados consistentes |

**Mensagem principal:** O modelo mais sofisticado do mundo **não consegue corrigir dados mal preparados**. O pré-processamento não é um "detalhe" – é a base de tudo.

# OS PASSOS DO PRÉ-PROCESSAMENTO



O que vamos fazer hoje (e vocês vão praticar em casa)

| Etapa                    | O que é                           | Por que fazer                                   |
|--------------------------|-----------------------------------|-------------------------------------------------|
| 1. Valores nulos         | Dados que estão faltando          | Modelos não funcionam com "vazio"               |
| 2. Escalonamento         | Colocar variáveis na mesma escala | Evitar que variáveis maiores "dominem" o modelo |
| 3. Variáveis categóricas | Transformar texto em números      | Modelos só entendem números                     |
| 4. Pipelines             | Organizar todas as etapas         | Evitar vazamento de dados e organizar o código  |



Vamos ver cada uma dessas etapas em detalhe.

# VALORES NULOS (MISSING DATA)



Como lidar com dados faltando?

Por que acontece?

- Erro na coleta de dados
- Pessoa não respondeu uma pergunta em um formulário
- Sensor falhou
- Dados corrompidos



O que fazer? Duas estratégias principais:

| Estratégia            | Como funciona                                           | Quando usar                                       |
|-----------------------|---------------------------------------------------------|---------------------------------------------------|
| Remover               | Eliminar linhas ou colunas com nulos                    | Poucos dados faltando                             |
| Preencher (Imputação) | Substituir nulos por algum valor (média, mediana, moda) | Muitos dados faltando, não quer perder informação |

Exemplo em código (estudo em casa):

```
# Remover linhas com nulos
df.dropna()

# Preencher com a média
df['idade'].fillna(df['idade'].mean())
```

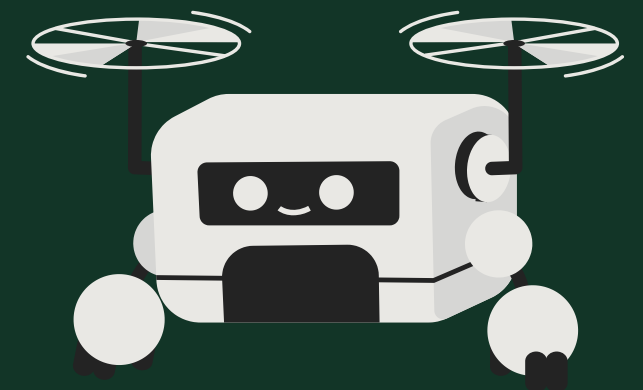


# ESCALONAMENTO DE DADOS

Por que colocar os dados na mesma escala?

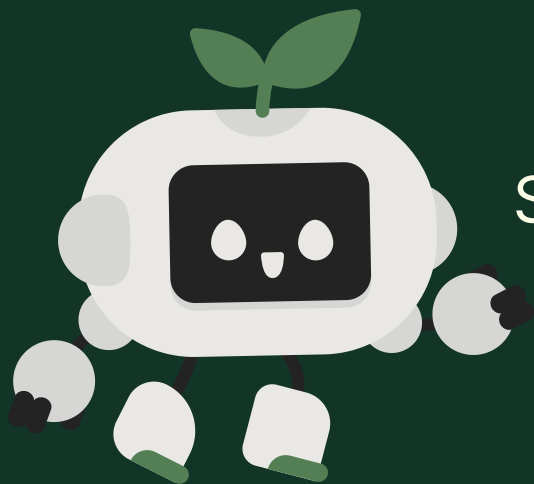
**O problema:** Variáveis em escalas diferentes podem distorcer o modelo.

| Variável         | Escala      | Exemplo de valor |
|------------------|-------------|------------------|
| Idade            | 0 a 100     | 25               |
| Renda anual      | 0 a 500.000 | 45.000           |
| Número de filhos | 0 a 10      | 2                |



Sem escalonamento, a **renda** (números grandes) vai ter muito mais peso que a **idade** ou **número de filhos**, mesmo que não seja importante para o problema.

**Solução:** Colocar todas as variáveis na mesma faixa de valores.



# MINMAXSCALER VS STANDARDSCALER



Os dois métodos mais comuns de escalonamento

| Método         | O que faz                                             | Fórmula intuitiva                                                  | Quando usar                                                           |
|----------------|-------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| MinMaxScaler   | Transforma dados para ficarem entre 0 e 1             | $(\text{valor} - \text{mínimo}) / (\text{máximo} - \text{mínimo})$ | Quando os dados <b>não</b> têm uma distribuição normal                |
| StandardScaler | Transforma dados para terem média 0 e desvio padrão 1 | $(\text{valor} - \text{média}) / \text{desvio padrão}$             | Quando os dados <b>têm</b> uma distribuição normal (ou próximo disso) |

**Exemplo:**

- **MinMaxScaler:** Valores de 18 a 65 anos viram valores entre 0 e 1
- **StandardScaler:** Idade média de 40 anos vira 0; 41 anos vira 0.1; 39 anos vira -0.1



# VARIÁVEIS CATEGÓRICAS

## Transformando texto em números

**O problema:** Modelos de Machine Learning só entendem números. Mas nossos dados frequentemente têm texto.

| Exemplo de dado bruto          | Não dá para o modelo entender |
|--------------------------------|-------------------------------|
| "Masculino" / "Feminino"       | ×                             |
| "São Paulo" / "Rio de Janeiro" | ×                             |
| "baixo" / "médio" / "alto"     | ×                             |



**Solução:** Converter texto em números de forma inteligente.





# ONE-HOT ENCODING

## One-Hot Encoding – O método mais comum

**Como funciona:** Criar uma coluna nova para cada categoria possível. Coloca 1 se a categoria está presente, 0 se não está.

**Exemplo:** Cidade = "São Paulo", "Rio de Janeiro", "Belo Horizonte"

| Original       | É_SãoPaulo | É_RioJaneiro | É_BeloHorizonte |
|----------------|------------|--------------|-----------------|
| São Paulo      | 1          | 0            | 0               |
| Rio de Janeiro | 0          | 1            | 0               |
| Belo Horizonte | 0          | 0            | 1               |

**Em código (estudo em casa):**

```
pd.get_dummies(df, columns=['cidade'])
```

**Cuidado:** Se a coluna tiver muitas categorias, isso pode criar muitas colunas novas (ex: 50 estados → 50 colunas). Nesse caso, existem outras estratégias.

# LABEL ENCODING (PARA VARIÁVEIS ORDINAIS)



**Label Encoding – Quando a ordem importa**

**Quando usar:** Quando as categorias têm uma ordem natural.

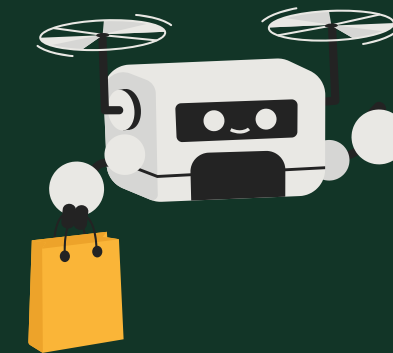


| Tamanho | Label Encoding |
|---------|----------------|
| Pequeno | 1              |
| Médio   | 2              |
| Grande  | 3              |

**Problema do Label Encoding em variáveis sem ordem:** O modelo pode entender que "Rio de Janeiro" (2) é maior que "São Paulo" (1) – o que não faz sentido.

**Regra prática:**

- **Sem ordem natural (cidades, cores, estados civis) → One-Hot Encoding**
- **Com ordem natural (tamanho, escolaridade, avaliação) → Label Encoding**



# PIPELINES - ORGANIZANDO TUDO

Pipelines – O passo a passo organizado

**Problema:** Fazer pré-processamento manualmente pode causar:

- Vazamento de dados (usar informações do teste no treino)
- Código bagunçado e difícil de repetir
- Erros difíceis de encontrar

**Solução:** Scikit-learn Pipelines



**Exemplo:**

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

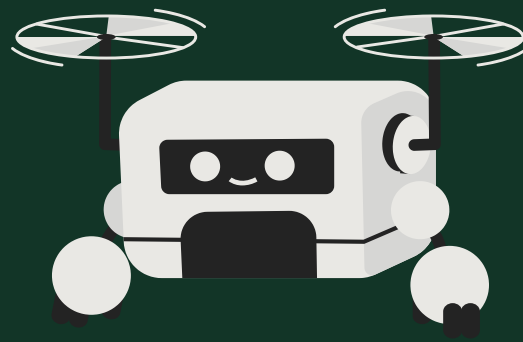
pipeline = Pipeline([
    ('scaler', StandardScaler()),      # passo 1: escalonar
    ('modelo', LogisticRegression())  # passo 2: treinar modelo
])

pipeline.fit(X_treino, y_treino)      # aplica os dois passos
```

**Vantagens:**

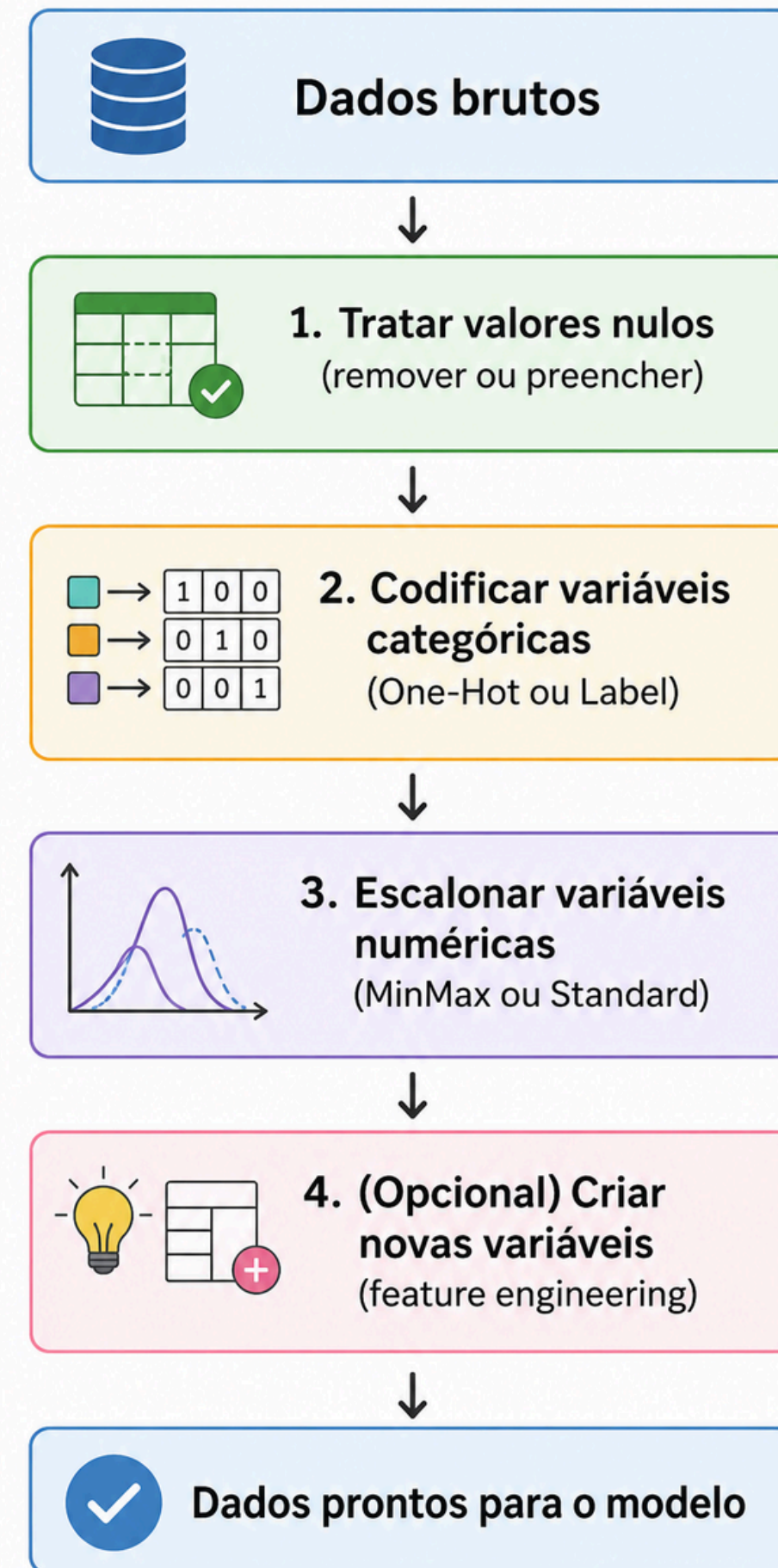
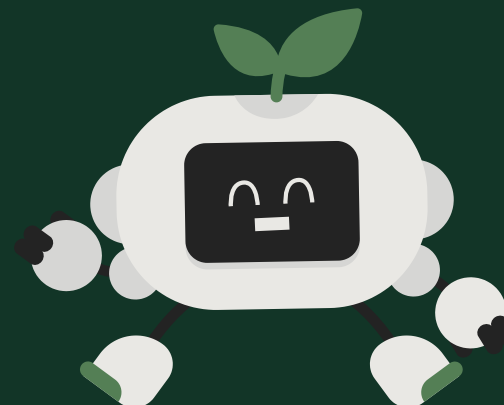
- Código mais limpo
- Evita vazamento de dados
- Fácil de testar diferentes combinações

2026



# RESUMO DO FLUXO DE PRÉ-PROCESSAMENTO

Esse fluxo é o que vocês vão aplicar na atividade em casa!



# EXEMPLO PRÁTICO

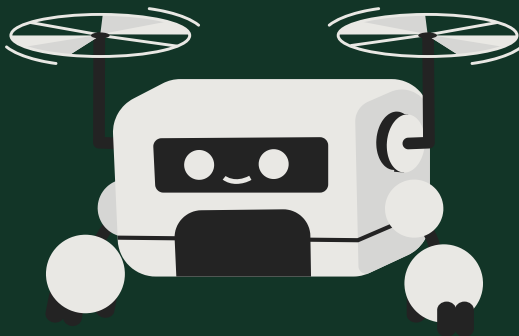
## Titanic – O que vamos fazer em casa

**Dataset:** Dados dos passageiros do Titanic (sobreviveu ou não)

**Problema:** Prever se um passageiro sobreviveu com base em:

- Idade
- Sexo
- Classe do ticket
- Porto de embarque
- Número de parentes a bordo

Etapas do pré-processamento no exemplo:



| Etapa                    | O que vamos aplicar                      |
|--------------------------|------------------------------------------|
| 1. Valores nulos         | Idade (mediana) / Porto (mais frequente) |
| 2. Escalonamento         | StandardScaler nas variáveis numéricas   |
| 3. Variáveis categóricas | One-Hot Encoding (sexo, porto)           |
| 4. Organização           | Pipeline do Scikit-learn                 |

**Resultado esperado:** Um modelo de classificação pronto para prever sobreviventes.



# O QUE VOCES VÃO TESTAR EM CASA



Depois que rodarem o código base, tentem mudar:

| O que mudar               | Como fazer                                           | Pergunta para responder              |
|---------------------------|------------------------------------------------------|--------------------------------------|
| Estratégia de imputação   | Trocar median por mean ou constant                   | Qual estratégia dá melhor resultado? |
| Método de escalonamento   | Trocar StandardScaler por MinMaxScaler               | Faz diferença no Titanic?            |
| Modelo                    | Trocar LogisticRegression por DecisionTreeClassifier | Qual modelo performa melhor?         |
| Ignorar pré-processamento | Remover o escalonamento ou o tratamento de nulos     | O que acontece com a acurácia?       |



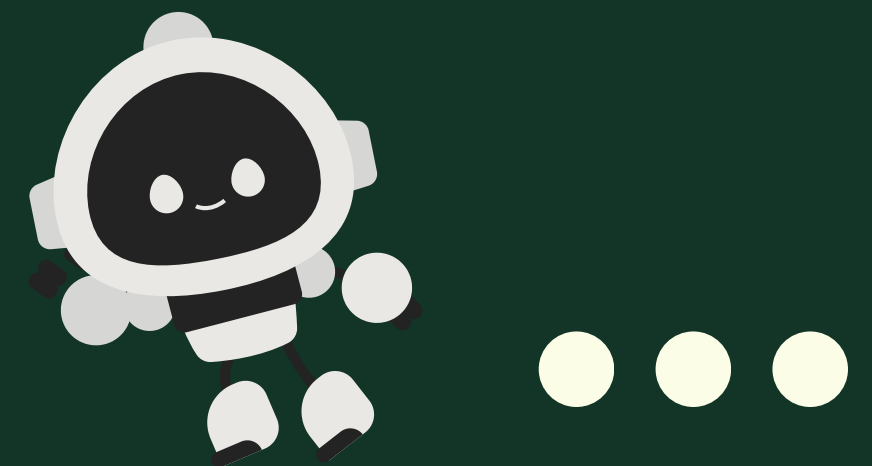
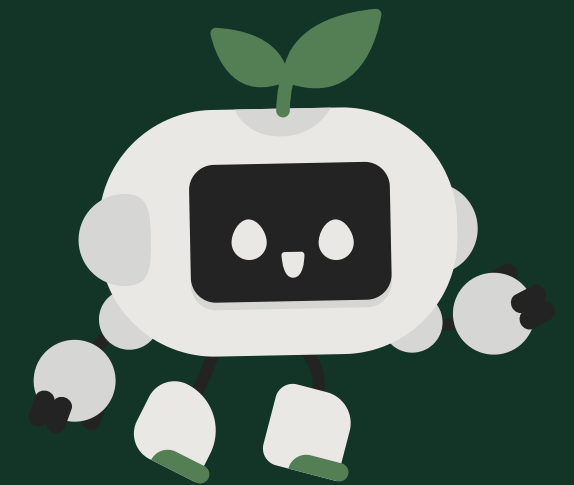
A ideia é experimentar e observar os resultados. Não existe resposta certa – o importante é aprender na prática.



# RESUMO DA SEMANA

## Recapitulando

- Pré-processamento é a etapa mais importante e mais demorada de um projeto de ML
- **Valores nulos:** remover ou preencher (média, mediana, moda)
- **Escalonamento:** colocar variáveis na mesma escala – MinMaxScaler ou StandardScaler
- **Variáveis categóricas:** transformar texto em números – One-Hot Encoding ou Label Encoding
- **Pipelines:** organizar o fluxo, evitar vazamento de dados e facilitar a vida
- Na atividade em casa, vocês vão aplicar tudo isso no dataset Titanic (código no repositório)



# PARA FAZER EM CASA

## Atividade para casa

**Objetivo:** Aplicar todas as etapas de pré-processamento em um dataset real.

### Instruções:

1. Baixar o notebook exemplo do repositório (dataset Titanic)
2. Executar o código completo – vejam se funciona
3. Testar as mudanças sugeridas no Slide 14
4. **Responder no notebook:**
  - Qual estratégia deu o melhor resultado? Por quê?
  - O que acontece se você NÃO escalonar os dados?
  - O que acontece se você não tratar os valores nulos?
5. Subir o notebook respondido na pasta da semana 12 do repositório





# REFERÊNCIAS

## Links úteis:

- [Documentação Scikit-learn – Pré-processamento](#)
- [Pipelines no Scikit-learn](#)
- [Dataset Titanic \(Kaggle\)](#)

